

JEPA- x : CROSS-PREDICTIVE PHYSICS GROUNDING FOR FORECASTABLE LATENT DYNAMICS

Kehan Wen¹, Ziming Li¹, Siyuan Luo¹, Fan Shi¹

¹Department of Electrical and Computer Engineering, NUS

ABSTRACT

Latent world models plan by predicting how candidate actions advance learned latent dynamics. In self-predictive models, however, the encoder and predictor are optimized jointly and can co-adapt to latent transitions that are easy to predict but weakly constrained by the physical evolution of the scene. We introduce the cross-predictive JEPA (JEPA- x), which grounds latent dynamics in privileged physical trajectories. JEPA- x treats visual observations and physical states as corresponding views of the same action-conditioned trajectory, advances both through a shared predictor, and matches each prediction to the future representations of both modalities. This encourages the action-conditioned predictor to learn a common transition rule across the two views. Privileged physical state is used only during training, leaving a visual-only model at deployment. Empirical results show that JEPA- x reduces the rollout drift of a newly fitted predictor from 0.361 to 0.104 and increases mean control success from 53.6% to 78.2% on a multi-task suite spanning six evaluation subfamilies. We additionally show that direct physical-state regression improves decodability without improving forecastability or control, indicating that the benefit comes from shaping latent dynamics rather than merely encoding physical variables.

1 INTRODUCTION

Planning with a latent world model casts control as search in representation space (Ha & Schmidhuber, 2018; Hafner et al., 2019; 2025; Hansen et al., 2024): the agent encodes the current scene, predicts the outcomes of candidate actions, and selects the actions whose predicted outcomes most closely approach the goal. This requires the world model to learn representations with reliable action-conditioned dynamics. In self-predictive models (LeCun, 2022; Assran et al., 2023; Zhou et al., 2024; Sobal et al., 2025), however, the encoder that defines the prediction target is optimized jointly with the predictor. The encoder and predictor can therefore co-adapt to a latent transition structure that is easy for the co-trained predictor to model. Although this produces temporal consistency within the learned latent space, it does not directly constrain the transitions to follow the physical evolution of the scene and potentially limits the model’s generalization capacity on unseen scenarios.

Privileged physical states have been widely used to shape learned representations and improve downstream behavior through state regression, distillation, cross-modal alignment, and training-time asymmetric critics (Chen et al., 2020; Gupta et al., 2016; Tian et al., 2020; Kumar et al., 2021; Pinto et al., 2018). Despite their differences, these approaches primarily use privileged information to supervise what a representation encodes or how it is used downstream. Our empirical results, however, show that making physical state decodable from visual latents does not necessarily make their action-conditioned evolution easier to predict. This gap reflects a distinction among three properties: decodability asks whether physical variables can be recovered from an individual latent state; forecastability asks whether the representation supports reliable action-conditioned prediction beyond the predictor with which it was trained; and control utility depends on whether predicted outcomes preserve the distinctions needed to rank candidate actions. Improving one property does not necessarily improve the others. We therefore treat visual observations and physical states as corresponding views of the same action-conditioned trajectory, allowing the physical future to constrain how visual representations evolve under action rather than only what physical variables an

individual latent state encodes. We then examine whether this trajectory-level constraint produces more forecastable visual dynamics and whether those dynamics improve control.

We introduce the *cross-predictive JEPA* (JEPA-*x*) to apply this trajectory-level constraint. The same action-conditioned predictor advances latent histories from either modality, and each prediction is matched to future representations in both modalities. The within-modal terms preserve self-prediction, while the cross-modal terms couple the predicted transitions across modalities. Sharing the predictor encourages the two encoders to expose a common action-conditioned transition rule rather than only align individual states. The physical branch is used only during training. At deployment, it is removed, leaving the same visual encoder–predictor architecture and planner as the visual-only baseline.

To test whether the improvement lies in the representation rather than its co-trained predictor, we freeze each visual encoder and train the same predictor from scratch. JEPA-*x* reduces rollout drift from 0.361 to 0.104 under this evaluation. A single model trained across multiple object–interaction pairs also raises mean control success from 53.6% to 78.2%, with improvements across all six evaluation subfamilies. In contrast, regression to the same physical state target improves decodability but leaves forecastability and control near the visual-only baseline. Ablations further show that correct visual–physical pairing is important for control, while direct cross-modal prediction improves forecastability beyond frame-level alignment.

In summary, our contributions are:

- **Cross-predictive JEPA.** We introduce a vision–physics prediction objective that uses a shared predictor to couple action-conditioned transitions across modalities, with privileged state required only during training.
- **A unified privileged-state interface.** We design a shared rigid-body representation that allows one physical encoder to operate across diverse manipulation scenes without task-specific labels.
- **More forecastable representations and better control.** By evaluating frozen representations with newly trained predictors, we show that JEPA-*x* improves visual-dynamics forecastability on the representation level. These gains lead to higher control success rate across the multi-task suite.

2 RELATED WORK

Predictive latent world models. World models differ in the structure that their learned representations preserve. Reconstruction-based approaches learn dynamics through pixels or generative latent variables (Ha & Schmidhuber, 2018; Hafner et al., 2019; 2020; 2025), whereas value-centric methods shape representations around reward and control objectives (Schrittwieser et al., 2020; Hansen et al., 2024). Representation-predictive methods instead model future features directly: DINO-WM plans in pretrained visual features (Zhou et al., 2024), PLDM learns latent dynamics from reward-free offline data (Sobal et al., 2025), and LeWM jointly learns visual representations and predictive dynamics (Maes et al., 2026). More broadly, joint-embedding objectives learn representations by predicting feature-space targets rather than reconstructing observations (LeCun, 2022; Assran et al., 2023; Bardes et al., 2023; 2024). Our work builds on this predictive paradigm and asks how corresponding physical trajectories can constrain the transition structure learned by a jointly optimized visual encoder and predictor.

Training-time privileged supervision. Learning with privileged information uses signals available during training but unavailable at deployment (Vapnik & Izmailov, 2015). Prior work transfers such information through distillation, cross-modal alignment, state regression, privileged sensing, or asymmetric actor–critic training (Chen et al., 2020; Lee et al., 2020; Gupta et al., 2016; Tian et al., 2020; Kumar et al., 2021; Pinto et al., 2018). These approaches establish several ways to exploit additional physical information during training, but our multi-task setting introduces a further requirement: the privileged representation must retain a consistent meaning across heterogeneous object–interaction configurations. We therefore represent scenes through a shared rigid-body schema based on object geometry, extent, pose, and effector state, allowing a single physical encoder to provide the privileged stream across all configurations without task-specific labels.

Physical grounding of predictive dynamics. Several recent world-model approaches bring privileged physical information closer to the learned dynamics. TWIST transfers state-based dynamics to an image-based student (Yamada et al., 2024), Scaffolder uses privileged sensors during policy learning (Hu et al., 2024), PIGDreamer aligns world-model representations with privileged information (Huang et al., 2025), and Pri4R introduces privileged 4D prediction during vision–language–action training (Kim et al., 2026). Closest to our setting, the concurrent Phys-JEPA imposes physical consistency on latent states and transitions for multivariate time-series forecasting (Nie et al., 2026), while PhyLatent adds training-only physical grounding and future-alignment objectives to the LeWM backbone (Zeng et al., 2026). JEPA-*x* instead treats privileged state as a second online predictive view: histories from either modality predict the corresponding future representations in both modalities, directly coupling visual–physical pairing with action-conditioned evolution. Unlike one-way distillation toward a fixed physical target, both representations remain online, and the privileged branch is removed entirely at deployment. We further study this coupling in the multi-task setting, where one model and one unified privileged-state interface span many object–interaction configurations.

3 PRELIMINARIES

Joint-embedding predictive architectures. (LeCun, 2022) Joint-embedding predictive architectures (JEPAs) learn representations by predicting a target embedding from an encoded context rather than reconstructing the target observation. In temporal settings, the context is a latent history and the prediction can be conditioned on actions. The encoder and predictor are trained jointly, with an additional mechanism used to prevent representational collapse.

LeWM. (Maes et al., 2026) LeWM applies this framework to action-conditioned visual prediction. A visual encoder maps each observation to $z_t^o = f_\theta(o_t)$, and a predictor estimates the future latent from a visual history Z_t^o and an action chunk a_t . Its training objective is

$$\mathcal{L}_{\text{LeWM}} = |g_\psi(Z_t^o, a_t) - z_{t+1}^o|_2^2 + \beta \Omega(z^o), \quad (1)$$

where Ω is the SIGReg isotropy regularizer (Balestriero & LeCun, 2025). For planning, candidate action sequences are rolled forward in the learned latent dynamics and scored against an encoded goal. We use LeWM as the visual-only baseline and retain its encoder, predictor, and planning procedure.

4 CROSS-PREDICTIVE JEPA

JEPA-*x* extends LeWM with a physical encoder and four within- and cross-modal prediction terms. The physical branch is used only during training and removed at deployment. Figure 1 provides an overview.

Core objective. For each visual trajectory, the simulator provides a synchronized sequence of privileged physical states. A physical encoder h_ϕ maps each state s_t to a latent representation

$$z_t^s = h_\phi(s_t),$$

with the same dimensionality as the visual latent z_t^o . The privileged state describes only the instantaneous scene configuration through the unified interface introduced below. It excludes velocities, contact forces, wrenches, and goal-relative quantities. Motion must therefore still be inferred from the state history and the applied actions.

Let Z_t^m denote a latent history from modality $m \in \{o, s\}$. Visual and physical histories are advanced by the same action-conditioned predictor g_ψ rather than separate modality-specific predictors. Given a history Z_t^m and an action chunk a_t , the predictor produces a future latent that is supervised by the future representations from both modalities. The complete objective is

$$\mathcal{L}_{\text{JEPA-}x} = \sum_{m,n \in \{o,s\}} |g_\psi(Z_t^m, a_t) - z_{t+1}^n|_2^2 + \beta [\Omega(z^o) + \Omega(z^s)], \quad (2)$$

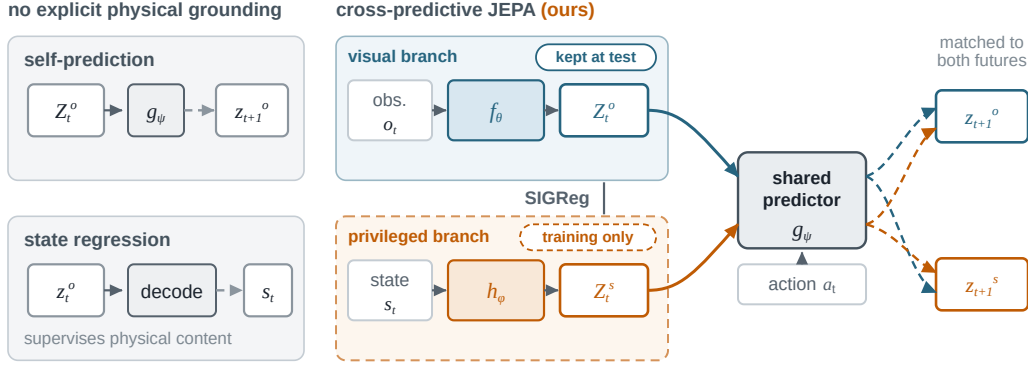


Figure 1: **Cross-predictive physical grounding constrains representation geometry and predictive dynamics.** Visual self-prediction jointly learns a visual encoder and predictor, while state regression makes physical variables decodable from individual visual latents. Neither directly couples a predicted visual transition to the corresponding physical future. JEPA- x instead treats visual and physical streams as paired views of the same action-conditioned trajectory and matches predictions from either history to future representations in both modalities. The physical branch is removed after training, leaving a visual-only model at deployment.

where actions modulate the predictor through AdaLN (Peebles & Xie, 2023). The isotropy regularizer Ω is applied independently to each branch.

The prediction loss contains four source–target terms. The visual-to-visual term is the original LeWM self-prediction objective. The physical-to-physical term learns predictive dynamics within the physical branch. The visual-to-physical term matches a transition predicted from visual history to the corresponding physical future, while the physical-to-visual term provides the reverse constraint. Using both cross-modal directions keeps the coupling symmetric.

Importantly, the two targets associated with a source history describe the same future scene reached under the same action. Thus, $g_\psi(Z_t^o, a_t)$ must match both z_{t+1}^o and z_{t+1}^s , and the same requirement applies to predictions from Z_t^s . The objective therefore couples the modalities through their action-conditioned evolution, rather than only aligning representations at individual time steps. Sharing g_ψ further encourages histories from both modalities to expose a common transition rule that can be advanced under the same actions.

All target representations are produced by the online encoders, and f_θ , h_ϕ , and g_ψ are optimized jointly. The physical branch is therefore not a fixed teacher or a predefined physical embedding. It adapts with the visual branch while remaining constrained by the structure of the privileged trajectory. Independent isotropy regularization prevents either branch from satisfying the prediction losses through representational collapse.

A unified privileged-state interface across tasks. Applying cross-modal grounding across heterogeneous tasks requires a consistent representation of privileged physical state. We therefore describe every scene using a unified rigid-body schema, as shown in Figure 2.

Each object contributes a token

$$\tau_i = [G_i \parallel e_i \parallel R_i \parallel t_i],$$

where G_i is a fixed-width descriptor of its canonical geometry (a pooled signed-distance field; Park et al., 2019), e_i contains its half-extents, and (R_i, t_i) specifies its pose. An effector token contains the end-effector rotation, position, and finger opening, while a learned table token completes the set. Scenes are padded to six object slots, with unused slots masked from both self-attention and pooling.

Because objects are represented by their geometry rather than their identity, the schema requires neither category labels nor task-specific state fields. A single physical encoder h_ϕ can therefore process every configuration in the suite. Appendix D provides the complete dimensions and normalization procedure.

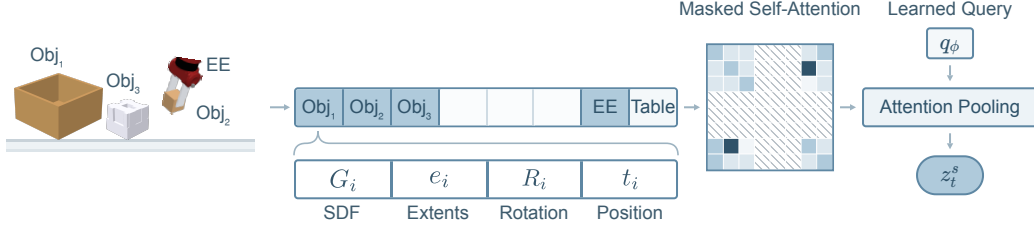


Figure 2: **Unified privileged-state interface.** A scene is represented by a fixed set of object slots, together with end-effector (EE) and table tokens; unused object slots are masked. Each object token encodes SDF geometry G_i , extents e_i , rotation R_i , and position t_i . Masked self-attention followed by learned-query attention pooling maps the resulting token set to the physical latent z_t^s .

The unified rigid-body schema provides a consistent physical description across configurations. This consistency enables cross-predictive coupling between visual and physical trajectories across tasks, allowing the shared predictor to learn common rigid-body and interaction dynamics without adapting to task-specific state formats.

Corresponding trajectories as a constraint on predictive evolution. The unified interface provides physical histories and futures paired with their visual counterparts at every time step. Corresponding cross-prediction couples this trajectory pairing to predictive evolution: histories from either modality predict future representations in both modalities. In this way, JEPA- x constrains how the shared representation geometry evolves under action rather than merely aligning visual and physical states at individual time steps.

Objective decomposition. The interaction between alignment and predictive evolution becomes explicit under uniform weighting of the four prediction terms. Define

$$p^m = g_\psi(Z_t^m, a_t), \quad \mu_{t+1} = \frac{1}{2} (z_{t+1}^o + z_{t+1}^s).$$

By the parallelogram identity,

$$\sum_{m,n \in \{o,s\}} \|p^m - z_{t+1}^n\|_2^2 = 2 \|p^o - \mu_{t+1}\|_2^2 + 2 \|p^s - \mu_{t+1}\|_2^2 + \|z_{t+1}^o - z_{t+1}^s\|_2^2. \quad (3)$$

Equation 3 exposes two consequences of cross-prediction over correctly paired trajectories. The final term encourages the visual and physical representations of the same future scene to align, while the first two require histories from either modality to predict that shared future under action. JEPA- x therefore combines cross-modal representation alignment with an action-conditioned constraint on how the shared latent geometry evolves. Because both targets are online encoder outputs, this coupling is symmetric rather than a one-way distillation toward a fixed physical target.

Deployment. Privileged state is used only during training. At deployment, the physical encoder h_ϕ is discarded, leaving the visual encoder and predictor (f_θ, g_ψ). The deployed model therefore receives the same observations and has the same inference architecture and computational cost as the visual-only baseline.

Candidate action sequences are rolled forward through the learned latent dynamics and scored by the distance between their predicted terminal latent and the encoded goal:

$$J(a_{1:T}) = \|\hat{z}_T(a_{1:T}) - f_\theta(o_g)\|_2^2, \quad \hat{z}_{i+1} = g_\psi(\hat{Z}_i, a_i). \quad (4)$$

The selected sequence is executed, and planning repeats from the resulting state. At test time, the effect of privileged state is carried entirely by the visual representation and predictive dynamics learned during training; privileged state is neither observed nor reconstructed during planning. More forecastable dynamics reduce one source of error in this rollout-based score, although successful control also requires latent distance to remain aligned with physical outcomes.

On the multi-task suite, candidates are drawn using z -CEM: a small unconditional variational autoencoder is trained on action chunks from the same corpus, CEM searches its latent space, and each candidate is decoded into a temporally coherent action chunk. Searching this space avoids

direct search over raw action sequences, which yields incoherent candidates far from the action distribution seen during training. The autoencoder observes neither the task nor the scene, so the same action prior serves every method. The single-task experiments retain LeWM’s released action-space CEM planner. The autoencoder specification and full planner hyperparameters are provided in Appendix E.

5 EXPERIMENTS

Setup. We evaluate on four single-task LeWM environments—*Push-T* (Chi et al., 2025), *OGBench-Block* (Park et al., 2025), *Two-Room*, and *Reacher* (Tassa et al., 2018)—using the released datasets, planners, and success predicates (Maes et al., 2026). We additionally evaluate on a multi-task tabletop suite built on Meta-World (Yu et al., 2020; Todorov et al., 2012), where a single model is trained jointly across 22 object–task configurations spanning 13 distinct assets. Control is evaluated on six evaluation subfamilies. Forecastability and decodability are measured on held-out episodes spanning every configuration. Appendix A gives the exact configuration counts and the mapping between configurations and evaluated families. Core comparisons share the training data, visual encoder, optimization schedule, and deployment planner; each ablation changes only the stated objective or architectural component.

We evaluate three properties. *Control Utility* is closed-loop task success. *Forecastability* measures how readily the learned representation supports action-conditioned prediction independently of its co-trained predictor. For each method, we freeze the visual encoder, discard the co-trained predictor, and fit the same lightweight action-conditioned predictor from scratch. Refitting the predictor separates representation forecastability from encoder–predictor co-adaptation during training. Refitting the predictor directly tests whether predictable transition structure is carried by the learned representation rather than only by the jointly optimized encoder–predictor pair. Because the same predictor family and fitting protocol are used for every frozen representation, differences in rollout error more directly reflect how readily each latent space supports action-conditioned prediction. We report rollout drift relative to a temporal-persistence baseline, with lower values indicating more forecastable dynamics. *Decodability* is measured by the R^2 of physical variables predicted from the frozen visual latent; we report the position of the manipulated object, and Appendix C specifies the full decoding target and the remaining groups. Appendices A and C provide the complete evaluation protocols.

JEPA- x improves forecastability. A fresh predictor models JEPA- x ’s action-conditioned dynamics more accurately than the baselines throughout the rollout. On the multi-task suite, relative rollout drift falls from 0.361 for VISUAL to 0.104 for JEPA- x . Figure 3 shows that the separation persists at every horizon, including beyond $h = 3$, when the rollout becomes fully autoregressive. Similar temporal-persistence errors and rank-matched PCA show that neither reduced temporal variation nor effective rank alone explains the gap (Appendix C).

The improvement is consistent across all four matched single-task environments (Table 1). Relative drift decreases from 0.503 to 0.221 on Two-Room, from 0.507 to 0.269 on OGBench-Block, from 0.399 to 0.258 on Push-T, and from 0.313 to 0.233 on Reacher.

JEPA- x improves multi-task control. On the multi-task suite, JEPA- x raises mean task success from 53.6% to 78.2%, a gain of 24.6 percentage points under the same planner, action prior, and candidate budget (Figure 4). The improvement holds across all six interaction families.

Figure 5 compares the two models’ predictions under identical action sequences. Because both models predict in latent space, we fit a separate state decoder, decode the manipulated-object and end-effector positions, and render the resulting rollouts alongside the ground truth. JEPA- x imagines

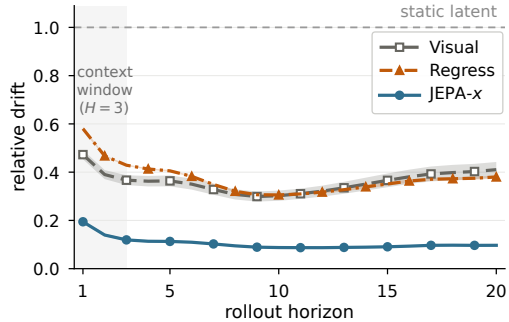


Figure 3: **JEPA- x produces more forecastable latent dynamics throughout the rollout.** Multi-task suite, three training seeds.

Table 1: **JEPA- x produces the most forecastable visual dynamics across all four matched single-task environments.** Forecastability is measured using a predictor fitted from scratch to each frozen representation. Object-position R^2 measures how well the environment’s object position is decoded from the frozen visual latent. Values are means $\pm$ sample SD over three training seeds. Bold marks the best mean in each comparison; exact ties are all bold.

Environment	Relative drift $\downarrow$			Object-position $R^2 \uparrow$		
	JEPA- x	VISUAL	REGRESS	JEPA- x	VISUAL	REGRESS
Two-Room	0.221$\pm$0.021	0.503 $\pm$ 0.010	0.480 $\pm$ 0.008	0.998$\pm$0.000	0.921 $\pm$ 0.011	0.936 $\pm$ 0.007
OGBench-Block	0.269$\pm$0.007	0.507 $\pm$ 0.026	0.393 $\pm$ 0.018	0.982 $\pm$ 0.003	0.879 $\pm$ 0.009	0.983$\pm$0.001
Push-T	0.258$\pm$0.007	0.399 $\pm$ 0.118	0.343 $\pm$ 0.039	0.991$\pm$0.001	0.944 $\pm$ 0.009	0.968 $\pm$ 0.007
Reacher	0.233$\pm$0.009	0.313 $\pm$ 0.009	0.309 $\pm$ 0.003	0.999$\pm$0.000	0.999$\pm$0.000	0.999$\pm$0.000

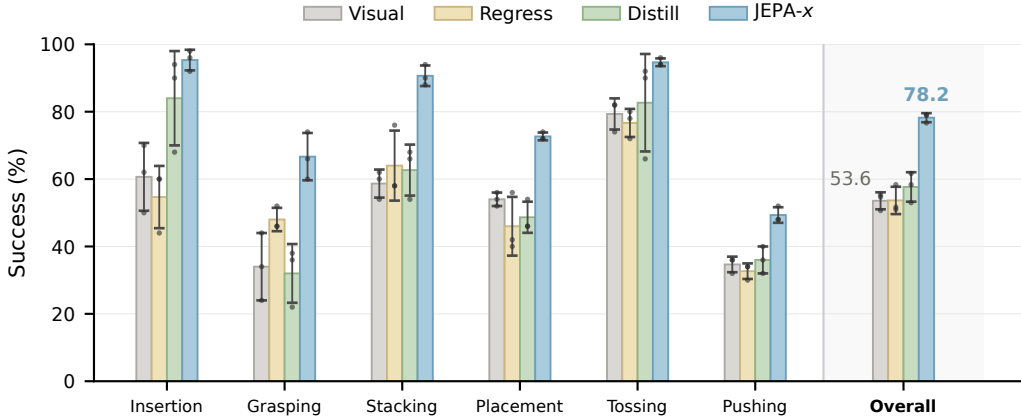


Figure 4: **Grounding improves multi-task control.** JEPA- x raises overall success from 53.6% to 78.2%, with improvements across all six interaction families. Regressing a comprehensive pose-based physical-state target from the visual latent does not recover the gain. Bars show means $\pm$ sample SD over three training seeds; dots show individual seeds.

the eraser arriving on top of the block, whereas VISUAL places it on the table beside the block. Across the three planning cycles of this episode, the decoded terminal position error averages 1.4 cm for JEPA- x and 3.1 cm for VISUAL (Appendix C). The decoded rollouts make the drift difference concrete: the same actions lead the two models to different imagined terminal configurations.

State regression does not recover these forecastability or control gains. REGRESS directly predicts a comprehensive pose-based target from the visual latent and raises manipulated-object position decodability to $R^2 = 0.991$, from 0.978 for both JEPA- x and VISUAL. Its control success remains at 53.7% and its relative rollout drift at 0.373. Position can therefore be readily available to a probe even when the latent transition remains difficult to forecast. The same holds when the privileged head is asked for the physical *future* rather than the present: an action-conditioned head predicting the next state, or its increment, from (z_t^o, a_t) leaves control at 54.4% and 51.3% and relative drift at 0.386 and 0.359, all within the seed spread of VISUAL (Appendix G). What distinguishes JEPA- x is therefore not that the target lies in the future, but that it lies in the physical branch’s learned representation. On the matched single-task environments, JEPA- x significantly improves control on Two-Room and OGBench-Block, while its mean success is within one percentage point of VISUAL on Push-T and Reacher (Table 2). Forecastability improves across all four environments, whereas the control gains are task dependent. This separation helps clarify the role of forecastability in planning. JEPA- x makes the latent dynamics easier to predict in all four environments, but the downstream control benefit remains task dependent. This pattern is consistent with rollout error mattering more when it changes the planner’s ranking of candidate actions, although the present evaluation does not isolate that mechanism. The single-task results therefore support a more limited claim: forecastable dynamics improve one component required by rollout-based planning, rather than guaranteeing higher control success on every task.

Correspondence and cross-prediction play distinct roles. We next separate the effects of correct visual–physical trajectory pairing, direct cross-modal prediction, and predictor sharing. CROSS-

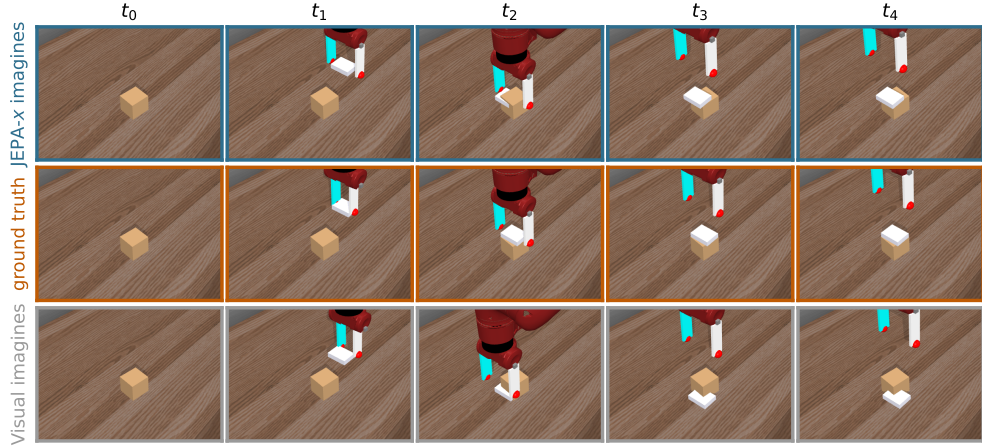


Figure 5: **Rolled over identical expert actions, JEPA-x imagines the eraser reaching the block; VISUAL imagines it beside the block.** Middle row: ground truth. Top and bottom: each model’s own rollout over the same action chunks from the same observation, with the decoded object and end-effector positions written back into the simulator and rendered.

Table 2: **Matched single-task control success.** Values are mean success rates (%) $\pm$ sample SD over three training seeds. Bold marks the best mean in each row; exact ties are all bold.

Environment	JEPA-x	VISUAL	REGRESS
Two-Room	94.8$\pm$1.0	74.7 $\pm$ 4.1	75.8 $\pm$ 3.8
OGBench-Block	78.7$\pm$2.7	64.9 $\pm$ 1.4	70.2 $\pm$ 0.8
Push-T	94.1 $\pm$ 0.4	94.9 $\pm$ 1.1	96.9$\pm$2.7
Reacher	83.6$\pm$1.4	82.7 $\pm$ 2.1	82.6 $\pm$ 0.4

ONLY retains cross-prediction but uses separate predictors for the visual and physical streams. SHARE-ONLY retains the shared predictor but removes the cross-modal prediction terms. ALIGN-ONLY replaces cross-modal prediction with symmetric frame-level alignment while retaining within-modality prediction. DISTILL replaces predictive coupling with pointwise regression onto a stop-gradient physical latent. Finally, SHUFFLE retains the complete JEPA-x architecture but uses a fixed corpus-wide derangement, assigning each visual trajectory a single intact but incorrect privileged partner throughout training.

Table 3 shows that cross-modal predictive coupling remains effective without predictor sharing, whereas predictor sharing alone does not recover the control gain. CROSS-ONLY reaches 73.0% control, recovering roughly four fifths of the gain from VISUAL to JEPA-x, with comparable rollout drift to JEPA-x. SHARE-ONLY reaches 56.7% control, close to the VISUAL baseline. Most of the control gain therefore survives without predictor-parameter sharing. Cross-modal predictive coupling remains effective with separate predictors, whereas sharing predictor parameters without cross-modal prediction produces only a small improvement over VISUAL.

ALIGN-ONLY recovers most of the control gain (72.6% against 78.2%) but has higher drift than JEPA-x (0.158 against 0.104), indicating that direct cross-modal prediction improves forecastability beyond pointwise alignment. Its aligned states are also trained with within-modality prediction, so it still carries cross-modal information forward indirectly. Across these variants, objectives using correctly paired visual-physical trajectories through alignment or cross-prediction retain most of the control improvement, while direct cross-modal prediction produces substantially lower rollout drift than frame-level alignment.

Pointwise distillation does not reproduce JEPA-x’s gains: DISTILL reaches 57.7% control and increases relative rollout drift to 0.516, compared with 0.361 for VISUAL. Together with REGRESS, this shows that making physical state decodable or matching it pointwise is not a substitute for grounding the transition itself.

SHUFFLE tests the importance of correct visual-physical correspondence while preserving a stable alternative pairing: each visual trajectory is assigned one intact but incorrect privileged trajectory

Table 3: **Correct trajectory pairing supports control, while direct cross-modal prediction improves forecastability beyond alignment.** The design columns indicate cross-modal prediction, predictor sharing, and correct visual-physical pairing (✓ present, × present but mispaired, – absent). Drift and control are means $\pm$ sample SD over three training seeds; bold marks the best mean in each column.

Variant	Cross-pred.	Shared	Corresp.	Drift ↓	Control ↑
JEPA- <i>x</i>	✓	✓	✓	0.104 $\pm$ 0.003	78.2$\pm$1.3
CROSS-ONLY	✓	–	✓	0.101$\pm$0.001	73.0 $\pm$ 0.9
SHARE-ONLY	–	✓	–	0.309 $\pm$ 0.020	56.7 $\pm$ 3.5
ALIGN-ONLY	–	✓	✓	0.158 $\pm$ 0.001	72.6 $\pm$ 3.6
DISTILL	–	–	✓	0.516 $\pm$ 0.004	57.7 $\pm$ 4.4
SHUFFLE	✓	✓	×	0.540 $\pm$ 0.023	3.6 $\pm$ 1.3
VISUAL	–	–	–	0.361 $\pm$ 0.024	53.6 $\pm$ 2.5

throughout training. Despite retaining the full predictive architecture and intact privileged trajectories, this fixed mispairing degrades both properties: control falls to 3.6% and relative drift rises to 0.540, above the 0.361 of VISUAL, which receives no privileged grounding. A consistent but incorrect pairing is therefore more damaging than omitting privileged grounding in this construction. Training against incompatible cross-modal targets disrupts the transition structure available to a freshly fitted predictor, rather than leaving the visual representation unaffected.

These ablations identify distinct contributions within cross-predictive grounding. Correct trajectory correspondence is necessary for cross-predictive grounding to improve rather than disrupt the visual representation: replacing the corresponding physical trajectory with a fixed incorrect partner degrades both forecastability and control below the ungrounded baseline. Given correct pairs, direct cross-modal prediction shapes action-conditioned evolution more effectively than frame-level alignment, reducing relative drift from 0.158 to approximately 0.10. Predictor sharing alone recovers neither benefit, while pointwise regression and distillation show that physical-state readout or latent matching is not a substitute for predictive coupling. Together, the results support the claim that the gain comes from constraining visual latent transitions with the corresponding physical future, rather than merely adding physical information or sharing model parameters.

6 DISCUSSION AND CONCLUSION

Across the multi-task suite, JEPA-*x* improves both fresh-predictor forecastability and closed-loop control, while direct state regression increases position decodability without comparable gains in either. Snapshot physical content therefore does not determine whether a representation supports reliable rollout prediction. JEPA-*x* instead couples visual latent evolution to corresponding physical trajectories. Because both encoders are learned jointly, this does not impose a canonical physical representation; privileged state provides a structured view of the same action-conditioned trajectory that constrains predictive evolution.

The ablations distinguish correct correspondence from the form of cross-modal coupling. Replacing each corresponding physical trajectory with a fixed incorrect partner degrades forecastability and control below the visual-only baseline, showing that correct pairing is essential for cross-predictive grounding. Given correct pairs, direct cross-modal prediction reduces rollout drift beyond frame-level alignment, whereas predictor sharing alone recovers little of the gain. Pointwise regression and distillation likewise fail to reproduce the forecastability improvement, locating the benefit in corresponding predictive coupling rather than parameter sharing or snapshot-level matching.

This distinction matters for rollout-based planning, where candidate actions are ranked by distances between predicted terminal latents and the goal. JEPA-*x* retains its lower drift after the co-trained predictor is replaced, indicating that the learned representation carries more forecastable transition structure. However, the single-task results show that improved forecastability does not guarantee higher success in every environment. It strengthens one component required for rollout-based planning, while its control benefit remains task dependent.

Our results are limited to simulation, paired privileged trajectories during training, and a fixed rigid-body state schema. Extending the approach to real-world observations and broader physical interactions remains future work.

REFERENCES

- Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15619–15629. IEEE, 2023.
- Randall Balestriero and Yann LeCun. LeJEPa: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.
- Adrien Bardes, Jean Ponce, and Yann LeCun. MC-JEPa: A joint-embedding predictive architecture for self-supervised learning of motion and content features. *arXiv preprint arXiv:2307.12698*, 2023.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024.
- Dian Chen, Brady Zhou, Vladlen Koltun, and Philipp Krähenbühl. Learning by cheating. In *Proceedings of the Conference on Robot Learning*, volume 100, pp. 66–75. PMLR, 2020.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- Saurabh Gupta, Judy Hoffman, and Jitendra Malik. Cross modal distillation for supervision transfer. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2827–2836, 2016.
- David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, pp. 2555–2565. PMLR, 2019.
- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025. doi: 10.1038/s41586-025-08744-2.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.
- Edward Hu, James Springer, Oleh Rybkin, and Dinesh Jayaraman. Privileged sensing scaffolds reinforcement learning. In *International Conference on Learning Representations*, 2024.
- Dongchi Huang, Jiaqi Wang, Yang Li, Chunhe Xia, Tianle Zhang, and Kaige Zhang. PIGDreamer: Privileged information guided world models for safe partially observable reinforcement learning. *arXiv preprint arXiv:2508.02159*, 2025.
- Jisoo Kim, Jungbin Cho, Sanghyeok Chu, Ananya Bal, Jinhyung Kim, Gunhee Lee, Sihaeng Lee, Seung Hwan Kim, Bohyung Han, Hyunmin Lee, et al. Pri4R: Learning world dynamics for vision-language-action models with privileged 4D representation. *arXiv preprint arXiv:2603.01549*, 2026.
- Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Proceedings of Robotics: Science and Systems*, 2021. doi: 10.15607/RSS.2021.XVII.011.

- Yann LeCun. A path towards autonomous machine intelligence. OpenReview, 2022. Version 0.9.2.
- Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47):eabc5986, 2020.
- Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- Weizhi Nie, Weichao Liu, Honglin Guo, and Yuting Su. Phys-JEPA: Physics-informed latent world models for multivariate time-series forecasting. *arXiv preprint arXiv:2606.16076*, 2026.
- Jeong Joon Park, Peter Florence, Julian Straub, Richard Newcombe, and Steven Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 165–174. IEEE, 2019.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. OGBench: Benchmarking offline goal-conditioned RL. In *International Conference on Learning Representations*, 2025.
- William Peebles and Saining Xie. Scalable diffusion models with transformers. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 4172–4182. IEEE, 2023.
- Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. In *Proceedings of Robotics: Science and Systems*, 2018. doi: 10.15607/RSS.2018.XIV.008.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588(7839):604–609, 2020.
- Uladzislau Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *Advances in Neural Information Processing Systems*, 38:43905–43941, 2025.
- Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. DeepMind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. In *European Conference on Computer Vision*, pp. 776–794. Springer, 2020.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033. IEEE, 2012.
- Vladimir Vapnik and Rauf Izmailov. Learning using privileged information: similarity control and knowledge transfer. *The Journal of Machine Learning Research*, 16(1):2023–2049, 2015.
- Jun Yamada, Marc Rigter, Jack Collins, and Ingmar Posner. TWIST: Teacher-student world model distillation for efficient sim-to-real transfer. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9190–9196. IEEE, 2024.
- Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-World: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Proceedings of the Conference on Robot Learning*, volume 100, pp. 1094–1100. PMLR, 2020.
- Xi Zeng, Haojie Ren, and Ziyang Song. PhyLatent: Learning dynamics-relevant representations for JEPA world models. *arXiv preprint arXiv:2608.05720*, 2026.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

A SUITE AND EVALUATION DETAILS

Suite composition. Our Meta-World-based corpus contains 22 object–task configurations over 13 distinct assets, with 450 clean episodes per configuration before noise augmentation. A single model is trained jointly across all configurations.

Table 4: The 22 configurations in the multi-task suite.

Interaction group	Configurations
Grasp–move–place	battery, block, can, tennis ball
Peg / plug insertion	peg-align-insert, peg-insert-pull, plug-insert
Planar pushing	block, book, eraser
Stacking	battery-on-phone, block-on-book, eraser-on-block
Righting / toppling	stand-up-can, stand-up-thermos, topple-battery, topple-can, topple-thermos
Tossing	battery, block, eraser, tennis ball

Success predicates. Success is determined from the executed physical trajectory rather than latent distance to the goal. Positional tolerances are asset-specific, and orientation is evaluated modulo each asset’s declared symmetry.

Table 5: Success predicates for the six evaluated interaction families.

Family	Success condition
Grasping	Object at least 5 cm above its initial table height over the final 1 s, with in-grasp slip at most 1 cm.
Stacking	Support contact between the stacked object and its base after a 2 s settle window, with the base orientation preserved.
Insertion	Insertion depth at least 90% of the target cavity depth, axis tilt at most 3°, and gripper open at termination.
Placement	Object at rest within the asset-specific positional tolerance of the target, with orientation compared modulo symmetry.
Pushing	Object within the asset-specific tolerance of the planar goal, with orientation compared to the initial pose modulo symmetry.
Tossing	Object inside the container interior at the end of the trajectory.

Scenario sampling. Each evaluated interaction family contains 50 held-out scenarios per training seed, giving 300 scenarios per model. Scenarios are stratified across configuration–subtask pairs and sampled with a fixed seed, so all methods are evaluated from the same initial states. The 300 scenarios are drawn from 266 source episodes; statistical tests therefore cluster segments originating from the same episode.

The interaction groups in Table 4 label corpus configurations, whereas the six evaluated families are subtask labels annotated within each episode. Scenarios are keyed by configuration–subtask pair, and each family’s quota is filled round-robin across every configuration containing that subtask. A grasp–move–place configuration therefore contributes segments to the grasping and placement families, the peg configurations to insertion, and the pushing, stacking and tossing configurations to their like-named families.

For the four LeWM environments, we use the benchmark’s released success predicates and evaluation loop unchanged. Each controller is evaluated on the same $N = 150$ paired scenarios.

B FULL CONTROL RESULTS

Per-family results. Table 6 reports the per-family results underlying Figure 4.

Statistical tests. For each training seed, suite comparisons use an episode-clustered paired sign-flip test with 10^5 Monte Carlo assignments. Scenarios originating from the same source episode share a sign. The per-seed JEPA- x –VISUAL differences are +28.3, +22.0, and +23.7 percentage points, while the respective JEPA- x –DISTILL differences are +17.3, +18.3, and +26.0 percentage points. All six seed-level comparisons yield $p_{MC} \leq 10^{-5}$, the smallest value 10^5 assignments can

Table 6: Per-family control success (%), mean $\pm$ sample SD over three training seeds. Families are ordered by the JEPA- x -VISUAL gap, matching Figure 4.

Family	JEPA- x	VISUAL	REGRESS	DISTILL	SHUFFLE
Insertion	95.3 $\pm$ 3.1	60.7 $\pm$ 10.1	54.7 $\pm$ 9.2	84.0 $\pm$ 14.0	16.0 $\pm$ 6.0
Grasping	66.7 $\pm$ 7.0	34.0 $\pm$ 10.0	48.0 $\pm$ 3.5	32.0 $\pm$ 8.7	1.3 $\pm$ 1.2
Stacking	90.7 $\pm$ 3.1	58.7 $\pm$ 4.2	64.0 $\pm$ 10.4	62.7 $\pm$ 7.6	0.7 $\pm$ 1.2
Placement	72.7 $\pm$ 1.2	54.0 $\pm$ 2.0	46.0 $\pm$ 8.7	48.7 $\pm$ 4.6	2.0 $\pm$ 2.0
Tossing	94.7 $\pm$ 1.2	79.3 $\pm$ 4.6	76.7 $\pm$ 4.2	82.7 $\pm$ 14.5	1.3 $\pm$ 2.3
Pushing	49.3 $\pm$ 2.3	34.7 $\pm$ 2.3	32.7 $\pm$ 2.3	36.0 $\pm$ 4.0	0.0 $\pm$ 0.0
All six	78.2$\pm$1.3	53.6 $\pm$ 2.5	53.7 $\pm$ 4.1	57.7 $\pm$ 4.4	3.6 $\pm$ 1.3

resolve. These tests quantify paired scenario-level differences for each trained model; consistency across training seeds is reported separately.

Single-task comparisons use exact McNemar tests on the same $N = 150$ paired scenarios. JEPA- x exceeds VISUAL in every seed on both Two-Room and OGBench-Block, with the largest p -value equal to 0.023.

For the comparison between JEPA- x and CROSS-ONLY, the training run is the unit of analysis. Mean suite control is 72.3/74.0/72.7 for CROSS-ONLY and 79.0/76.7/79.0 for JEPA- x .

C FORECASTABILITY PROTOCOL AND ADDITIONAL RESULTS

Fresh-predictor probe. To measure representation forecastability independently of the transition model used during representation learning, we freeze each encoder, cache its latents, discard the original predictor, and train a new predictor from scratch. We use the same lightweight probe family across methods, equalizing predictor capacity so that differences primarily reflect predictive structure in the learned representation rather than the co-trained transition model.

For the single-task experiments, the probe is action-conditioned. Its input concatenates the $H = 3$ most recent latent frames with the aligned $H = 3$ action chunks. A three-layer MLP of width 512 with GELU activations predicts the next latent as a residual update to the most recent frame. Latents are standardized per dimension using statistics from the fitting episodes; actions are used as stored. We train with AdamW at learning rate 10^{-3} and weight decay 10^{-5} for 30,000 steps, with batch size 256 on 1000 episodes and a randomized fit/evaluation episode split.

At evaluation, the probe is rolled out autoregressively for 20 steps, feeding back its own predictions while consuming the recorded action sequence. The resulting error therefore measures prediction under actions rather than latent smoothness alone. Normalization by a constant-latent baseline further removes credit for temporal persistence.

The multi-task results in Table 7 and Figure 3 use the same probe family with a direct rather than residual output head, 20,000 training steps, batch size 512, cosine learning-rate decay, and an episode-level fit/evaluation split.

Probe fitting and evaluation always use disjoint episodes. Relative drift is computed independently at each rollout horizon and normalized by a constant-latent predictor:

$$\text{drift}(h) = \frac{\mathbb{E}\|\hat{z}_{t+h} - z_{t+h}\|_2}{\mathbb{E}\|z_t - z_{t+h}\|_2}. \quad (5)$$

The reported aggregate is the mean of these per-horizon ratios over $h = 1, \dots, 20$.

Manipulated-object decoding probe. Decodability is measured using the same frozen encoder checkpoints and episode-level split as the dynamics probe, so both columns of Table 7 evaluate the same representations under matched data partitions. A single three-layer MLP of width 512 with GELU activations maps one frozen latent frame to physical variables; it is trained with AdamW at learning rate 10^{-3} and weight decay 10^{-5} for 6000 steps at batch size 1024, and evaluated on the held-out episodes. The decoding target is a 15-dimensional vector comprising the manipulated object’s normalized position, its orientation as a flattened 3×3 rotation matrix, and the normalized end-effector position. We compute $R^2 = 1 - \text{SSE}/\text{SST}$ separately for each target group over all

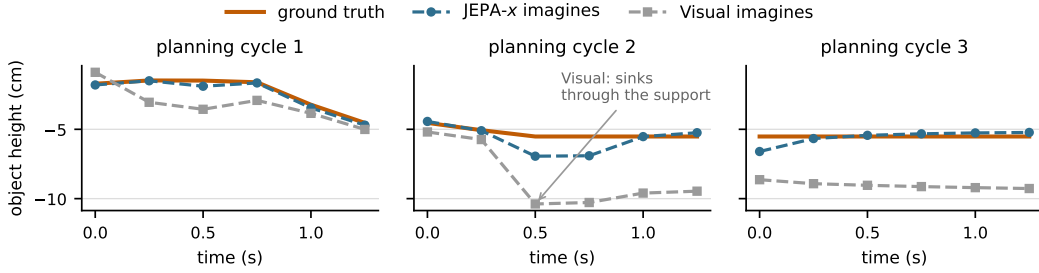


Figure 6: **Under the same actions, JEPA- x imagines the object coming to rest on its support.** Decoded height of the manipulated object during a stacking episode: ground truth against each model’s own rollout over the identical expert action chunks, one panel per planning cycle. VISUAL’s imagined object passes through the support and stays there; JEPA- x has a mean terminal error of 1.4 cm across the three cycles.

held-out frames, with the total sum of squares taken about the held-out mean. The tables report R^2 for the object-position group. Canonical geometry, bounding-box extents, and masked object slots are not included in the decoding target.

Multi-task forecastability. Table 7 reports the quantities underlying the multi-task comparison. Under the controlled fresh-predictor protocol, JEPA- x has lower raw rollout error and normalized drift than the visual-only and state-regression baselines. REGRESS and DISTILL make manipulated-object position more decodable, but neither substantially reduces rollout drift relative to VISUAL. In contrast, JEPA- x reduces drift without increasing the reported position decodability over VISUAL. SHUFFLE increases drift beyond every other variant and collapses control, showing that incorrect visual–physical pairing degrades forecastability and control together. Decodability and forecastability are therefore distinct properties, and the ordering of one does not predict the ordering of the other.

Table 7: Forecastability and manipulated-object position decodability on the multi-task suite. Both metrics use the same frozen encoders and held-out split, with a separately fitted probe for each metric. Drift and R^2 are means $\pm$ sample SD over three training seeds; bold marks the best reported mean. Raw and Copy are averaged over horizons and seeds; Drift is the mean of the per-horizon ratios and therefore differs from their ratio.

Arm	Raw $\downarrow$	Copy	Drift $\downarrow$	Manip.-object pos. $R^2 \uparrow$
JEPA- x	1.18	12.17	0.104$\pm$0.003	0.978 $\pm$ 0.002
VISUAL	4.43	12.42	0.361 $\pm$ 0.024	0.978 $\pm$ 0.000
REGRESS	4.28	12.07	0.373 $\pm$ 0.005	0.991$\pm$0.000
DISTILL	6.11	12.50	0.516 $\pm$ 0.004	0.991$\pm$0.001
SHUFFLE	8.41	15.51	0.540 $\pm$ 0.023	0.958 $\pm$ 0.002

SHUFFLE has both the largest raw rollout error (8.41, against 4.43 for VISUAL) and the largest Copy denominator (15.51, compared with approximately 12 for the other methods): the mispaired objective spreads its latents further apart while predicting them less well. Its manipulated-object decodability is also the lowest of any arm (0.958). Incorrect visual–physical pairing therefore degrades snapshot content, forecastability and control at once, rather than trading one against another.

Imagined trajectories under a fixed action sequence. Relative drift summarizes forecastability as a scalar; Figure 6 shows what it looks like on one trajectory in physical units. We take a stacking episode and, at the start of each planning cycle, roll each model’s predictor from the same observation over the same recorded expert action chunks. The manipulated object’s position is decoded from the predicted latents by a probe fitted on training episodes only ($R^2 = 0.984$ and 0.988 for the two models); first-step decoding error remains below 2 cm in every cycle. With identical actions and starting observations, the plots compare decoded rollout behavior; residual state-decoder error remains. Mean terminal error across the three cycles is 1.4 cm for JEPA- x and 3.1 cm for VISUAL.

Rank-controlled forecastability. Relative drift compares a fitted predictor against a temporal-persistence baseline, so a representation of lower intrinsic dimension could in principle be easier for a fixed-capacity probe to fit. JEPA- x does produce a lower-dimensional latent: measured as the participation ratio of the eigenspectrum, its effective rank is 56.5 ± 0.2 against 95.7 ± 2.4 for VISUAL,

out of 192 dimensions, and this holds for every training seed. To separate dimensionality from content we project each representation onto its leading k principal components and refit the identical probe. The basis is fitted on the training episodes only and then applied to all episodes, so the held-out split does not enter the projection. Everything else is held fixed: the same episodes, split, actions, probe architecture, and optimization budget, with three probe seeds per condition.

Table 8: **Matching effective rank does not close the forecastability gap.** Relative drift after projection onto the leading k principal components, reported as mean $\pm$ sample SD over three training seeds.

Projection	Effective rank		Relative drift $\downarrow$	
	JEPA- x	VISUAL	JEPA- x	VISUAL
None (native)	56.5	95.7	0.104$\pm$0.004	0.359 $\pm$ 0.018
$k = 57$	51.7	52.8	0.121 $\pm$ 0.008	0.453 $\pm$ 0.022
$k = 32$	30.7	30.4	0.175 $\pm$ 0.011	0.402 $\pm$ 0.031
$k = 16$	15.7	15.6	0.193 $\pm$ 0.011	0.379 $\pm$ 0.025
$k = 8$	7.9	7.9	0.214 $\pm$ 0.016	0.407 $\pm$ 0.033

At $k = 57$, chosen to approximate JEPA- x ’s native effective rank, VISUAL and JEPA- x reach measured ranks of 52.8 and 51.7, with drift of 0.453 and 0.121, respectively. Under more aggressive projection both arms degrade and the ratio narrows, from $3.5\times$ at native width to $1.9\times$ at rank 7.9, but the gap never closes. The retained variance shows why the two arms respond differently: at $k = 57$, JEPA- x retains 95.1% of its variance while VISUAL retains 67.9%, so the same nominal width discards far more of the visual-only representation. Compression never improves the visual baseline, and JEPA- x at $k = 8$ remains more forecastable than VISUAL at any tested width. The forecastability gap is therefore not explained by effective rank alone.

D PRIVILEGED-STATE INTERFACE

This section gives the exact dimensions and normalization of the privileged-state interface introduced in Section 4. Each object token concatenates a fixed-width canonical-geometry descriptor, bounding-box half-extents, a flattened 3×3 rotation matrix, and position. Our experiments use an 8^3 average-pooled signed-distance field as the geometry descriptor. The effector token carries end-effector rotation, position, and finger opening, while a learned table token completes the set. Scenes are padded to six object slots with unused slots masked from both self-attention and pooling, giving eight tokens in total. All physical quantities are normalized using training-set statistics.

The privileged input describes only the instantaneous physical configuration and excludes velocities, wrenches, contact forces, and goal-relative quantities. Both branches receive length- H histories, so they infer motion from temporal context and action.

E IMPLEMENTATION DETAILS

Encoders. The visual encoder is a ViT-Tiny (Dosovitskiy et al., 2021) with 192-dimensional features, patch size 14, and 224×224 input resolution (5.50M parameters). The physical encoder is a three-layer, four-head Transformer of width 192 with hidden width 256 (1.55M parameters), consuming the token set defined in Appendix D.

Predictor and objective. The action-conditioned predictor is a six-layer, 16-head Transformer operating on $H = 3$ latent frames, with head dimension 64, feed-forward width 2048, and dropout 0.1. Actions enter through zero-initialized AdaLN. Each action chunk spans five environment steps with a one-step prediction offset. The four source–target terms in Equation 2 receive equal weight. SIGReg is applied separately to the two branches with weight $\beta = 0.09$, 17 knots, and 1024 projections.

Both future latents are online encoder outputs; the JEPA- x prediction path contains no stop-gradient, exponential-moving-average encoder, or separate target network.

Optimization. Core matched methods use AdamW with learning rate 7×10^{-5} and weight decay 10^{-3} , batch size 256, bfloat16 precision, gradient clipping at 1.0, and a linear-warmup cosine-

annealing learning-rate schedule. All multi-task models are trained for 40 epochs over the augmented corpus. No image augmentation is applied at training time; input diversity comes from the corpus-level action-noise augmentation described under *Data* below. Training-time auxiliary components differ only where required by the stated objective or ablation.

Planning. The multi-task experiments use a horizon of five macro-steps, each containing five environment actions, with 300 candidates, 30 CEM refinement iterations, and 30 elites (10%). The search distribution is initialized at zero mean and unit variance in the latent action space and re-fitted to the elites at each iteration, with the current mean always evaluated as one candidate. The planner executes the entire five-macro-step sequence, that is 25 environment actions, before replanning from the resulting state; the executed segment is therefore open-loop and planning is repeated once per segment rather than at every macro-step. The unconditional action autoencoder used by *z*-CEM is a variational autoencoder with a 32-dimensional latent whose encoder and decoder are two-hidden-layer MLPs of width 512, mapping a flattened 125-dimensional action chunk to and from the latent. It is trained on the same corpus for 20,000 steps with AdamW at learning rate 10^{-3} , weight decay 10^{-5} , and cosine decay, under a reconstruction plus KL objective with KL weight $\beta = 0.1$. It receives neither observations nor goals, and the same action prior is used for every method.

Directly sampling raw action sequences can produce temporally incoherent candidates far from the action distribution seen during training. Searching in the autoencoder latent space instead provides temporal structure: each latent candidate decodes to a coherent action chunk, and CEM refines a distribution over latent codes without conditioning candidates on the current task or scene.

Single-task experiments use the released LeWM action-space CEM planner and its original evaluation budget.

Data. The multi-task corpus is the union of one clean shard and four action-noise shards, giving 29,260 episodes and approximately 1.44M transitions across the 22 configurations. The clean shard contributes 9,900 episodes (450 per configuration) collected with noise disabled. The four noise shards contribute 5,104, 4,752, 4,752 and 4,752 episodes and differ only in noise scale.

Noise is injected into the expert controller during collection rather than added to a recorded trajectory, so every noisy episode is a physically consistent rollout of a perturbed policy. It acts at two levels. At each replanning tick the target waypoint is displaced by $\mathcal{N}(0, \sigma_p^2 I_3)$ and the target yaw by $\mathcal{N}(0, \sigma_{yaw}^2)$; the displacement is held fixed for the whole replan segment, so replanning cannot average it away, and the perturbed target is then clipped into the reachable workspace. At every control step an Ornstein–Uhlenbeck process perturbs the action itself,

$$x \leftarrow (1 - \theta)x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_{\text{step}}^2 s^2), \quad \sigma_{\text{step}} = \sigma_a \sqrt{2\theta - \theta^2}, \quad (6)$$

with $\theta = 0.15$ and per-dimension scale $s = [1, 1, 1, 1, 0.25]$, so the gripper channel receives a quarter of the amplitude. The step variance is chosen so the stationary per-dimension standard deviation equals σ_a . The sample is added to the controller’s output and the sum is clipped to $[-1, 1]$.

The σ values carry a per-family calibration factor k , fitted by bisection so the expert’s failure rate on that family lands in a target band. The four shards are the resulting tiers: the band shard targets a 20–30% failure rate, and the s50, s20 and s0 shards target 45–55%, 75–85% and at least 98%, giving $k = 0.0625, 0.0859, 0.1094$ and 0.325 with (σ_p, σ_a) of $(2.4 \times 10^{-4}, 6.3 \times 10^{-3}), (3.3 \times 10^{-4}, 8.6 \times 10^{-3}), (4.2 \times 10^{-4}, 1.1 \times 10^{-2})$ and $(1.2 \times 10^{-3}, 3.3 \times 10^{-2})$. Yaw waypoint noise is disabled throughout ($\sigma_{yaw} = 0$). Noise is therefore graded by how much it degrades the expert, not by a fixed magnitude, and the corpus spans from a lightly perturbed expert to one that essentially never succeeds.

The LeWM datasets are used as released. Code, the corpus-generation pipeline, and evaluation harnesses will be released.

F ABLATION DETAILS

All ablations use the same training data, visual backbone, latent dimensionality, optimization hyperparameters, and training schedule as JEPA-*x*. Predictor parameterization and cross-modal prediction terms are changed only where required by the intervention being tested.

CROSS-ONLY. CROSS-ONLY retains all four source–target prediction terms from JEPA- x and therefore preserves corresponding cross-prediction, but replaces the shared predictor with separate visual and physical predictors. This removes predictor-parameter sharing while preserving corresponding cross-prediction and direct cross-modal transition coupling.

SHARE-ONLY. SHARE-ONLY retains the shared action-conditioned predictor but removes the two cross-modal prediction terms. Each modality is therefore trained only to predict its own future representation through the common predictor. This isolates predictor sharing without directly grounding either prediction in the future of the other modality.

REGRESS. REGRESS keeps the visual branch and its self-prediction term unchanged and adds a per-frame state-regression head. A two-layer MLP of width 512 maps each visual latent to an 85-dimensional target formed by six object slots, each contributing a flattened 3×3 rotation matrix and a position, followed by the effector’s rotation, position, and finger opening:

$$\mathcal{L}_{\text{reg}} = \frac{\sum_d m_d ([r_\omega(z_t^o)]_d - s_{t,d})^2}{\sum_d m_d}, \quad (7)$$

where r_ω is the regression head and m_d masks the dimensions of unoccupied object slots; the effector dimensions are always included. Targets use the same training-set normalization as the privileged branch. The term is added to the visual self-prediction and isotropy losses with weight 1.0. No physical encoder or cross-modal prediction is present, and the regression head is unused at deployment.

DISTILL. DISTILL replaces predictive cross-modal coupling with pointwise regression. Each branch retains its own within-modality prediction term and is advanced by its own predictor, each followed by its own output projection that maps the predictor’s hidden state back to the embedding dimension. These projections belong to the prediction path only: they are applied to predictor outputs and play no part in the cross-branch coupling. Both within-modality prediction terms ($m = n$ in Equation 2) therefore remain active, and the physical encoder is still trained, by its own prediction term together with the isotropy regularizer. The two modalities are coupled only by aligning the raw encoder outputs, with no projection applied on either side, the visual latent being regressed onto a stop-gradient physical latent:

$$\|z_t^o - \text{sg}(z_t^s)\|_2^2. \quad (8)$$

The physical target remains paired with its observation, so this baseline preserves sample-level correspondence while constraining representation content rather than cross-modal predictive transitions.

ALIGN-ONLY. ALIGN-ONLY keeps the shared predictor and both branches’ within-modality prediction terms but excludes the two cross-modal terms from the loss, and adds an explicit symmetric alignment

$$\lambda_{\text{align}} \|z^o - z^s\|_2^2, \quad (9)$$

with $\lambda_{\text{align}} = 1$. The term is applied to the full encoded window rather than only to the prediction targets, and neither branch is detached.

SHUFFLE. SHUFFLE retains all four JEPA- x source–target terms and the shared predictor, but reads the privileged stream from a fixed partner episode:

$$Z_t^s \leftarrow Z_t^{s,\pi(e)}, \quad z_{t+1}^s \leftarrow z_{t+1}^{s,\pi(e)}, \quad (10)$$

where e is the current episode and π is a derangement of the training episodes, drawn once before training and held fixed. No episode is paired with itself, and each visual trajectory sees the same incorrect partner throughout training, so the mis-pairing is a consistent alternative pairing rather than a fresh random pairing at every step. Partners are drawn across the whole corpus, so a visual trajectory may be paired with a privileged trajectory from a different configuration; the privileged geometry, slot occupancy, and pair-validity masks follow the partner, and the partner window is taken at the same relative position within the partner episode. Because context and future are read from the same partner, each privileged sequence remains an intact trajectory that obeys the environment dynamics, and the isotropy regularizer operates on these intact trajectories as the privileged branch’s own

task. Pixels and actions stay on the true episode. The intervention therefore preserves the physical trajectories themselves while replacing their pairing with the visual stream by a fixed, incorrect one.

G PREDICTING THE PHYSICAL FUTURE IN RAW STATE SPACE

The closest alternative to cross-predictive grounding is to keep the visual model unchanged and add an action-conditioned head that predicts the physical future directly. Visual self-prediction and SIGReg are exactly REGRESS’s; what changes is the privileged supervision, which becomes an MLP mapping (z_t^o, a_t) to the next privileged state rather than the current one. Only the target space then separates this arm from JEPA- x : the raw physical future here, the physical branch’s representation of that future there. Deployment is pixel-only, identical to VISUAL and REGRESS; the head is unused.

We train two variants for three seeds each at the same budget. FUT-ABS regresses s_{t+1} . FUT-DELTA regresses $s_{t+1} - s_t$, which is the sharper test: measured on the corpus in the model’s normalized units, s_t already accounts for about 99% of the squared norm of s_{t+1} – most of the 85-dimensional target is rotation, which barely moves across a 250 ms row – so an absolute head can score well by decoding the present state, which is what REGRESS already supervises.

Table 9: Per-family control success (%) for the two future-state auxiliary variants, with VISUAL and REGRESS for reference. Mean $\pm$ sample SD over three training seeds, same protocol as Table 6.

Family	FUT-DELTA	FUT-ABS	VISUAL	REGRESS
Insertion	54.0 $\pm$ 6.9	56.7 $\pm$ 22.7	60.7 $\pm$ 10.1	54.7 $\pm$ 9.2
Grasping	39.3 $\pm$ 19.4	40.0 $\pm$ 6.9	34.0 $\pm$ 10.0	48.0 $\pm$ 3.5
Stacking	60.0 $\pm$ 2.0	60.0 $\pm$ 5.3	58.7 $\pm$ 4.2	64.0 $\pm$ 10.4
Placement	52.0 $\pm$ 4.0	56.0 $\pm$ 4.0	54.0 $\pm$ 2.0	46.0 $\pm$ 8.7
Tossing	72.7 $\pm$ 4.6	77.3 $\pm$ 6.4	79.3 $\pm$ 4.6	76.7 $\pm$ 4.2
Pushing	30.0 $\pm$ 3.5	36.7 $\pm$ 8.1	34.7 $\pm$ 2.3	32.7 $\pm$ 2.3
All six	51.3 $\pm$ 4.8	54.4 $\pm$ 3.1	53.6 $\pm$ 2.5	53.7 $\pm$ 4.1

Neither variant separates from the visual baseline on any of the three measurements. Control is 51.3 $\pm$ 4.8 for FUT-DELTA and 54.4 $\pm$ 3.1 for FUT-ABS, against 53.6 $\pm$ 2.5 for VISUAL and 53.7 $\pm$ 4.1 for REGRESS: the spread across variants is smaller than the spread across seeds. Relative rollout drift is 0.359 $\pm$ 0.010 and 0.386 $\pm$ 0.005, against 0.361 $\pm$ 0.024 for VISUAL and 0.104 for JEPA- x . Manipulated-object position decodability is 0.978 $\pm$ 0.001 and 0.986 $\pm$ 0.000; only the absolute variant rises above VISUAL’s 0.978, in the direction of REGRESS’s 0.991, which is what its present-state-heavy target predicts.

Predicting the physical future is therefore not sufficient on its own, and the target space is what distinguishes this arm from JEPA- x . Supervising the raw next state leaves forecastability and control at the visual baseline whether the head is asked for the state itself or for its increment; the gains reported in Section 5 require predicting the future *in the physical branch’s learned representation*, through the shared predictor.

H QUALITATIVE ROLLOUTS

Figures 7 and 8 show paired qualitative comparisons between JEPA- x and VISUAL across all six evaluated interaction families. For each family, both methods are executed from the same initial state using the same planner. We show a representative scenario on which the methods disagree, with JEPA- x succeeding and VISUAL failing. Frames are cropped to the working volume because the manipulated object occupies only a small fraction of the uncropped camera view, making criteria such as a 5 cm lift difficult to inspect at print scale. Quantitative comparisons are reported in Figure 4 and Table 6.

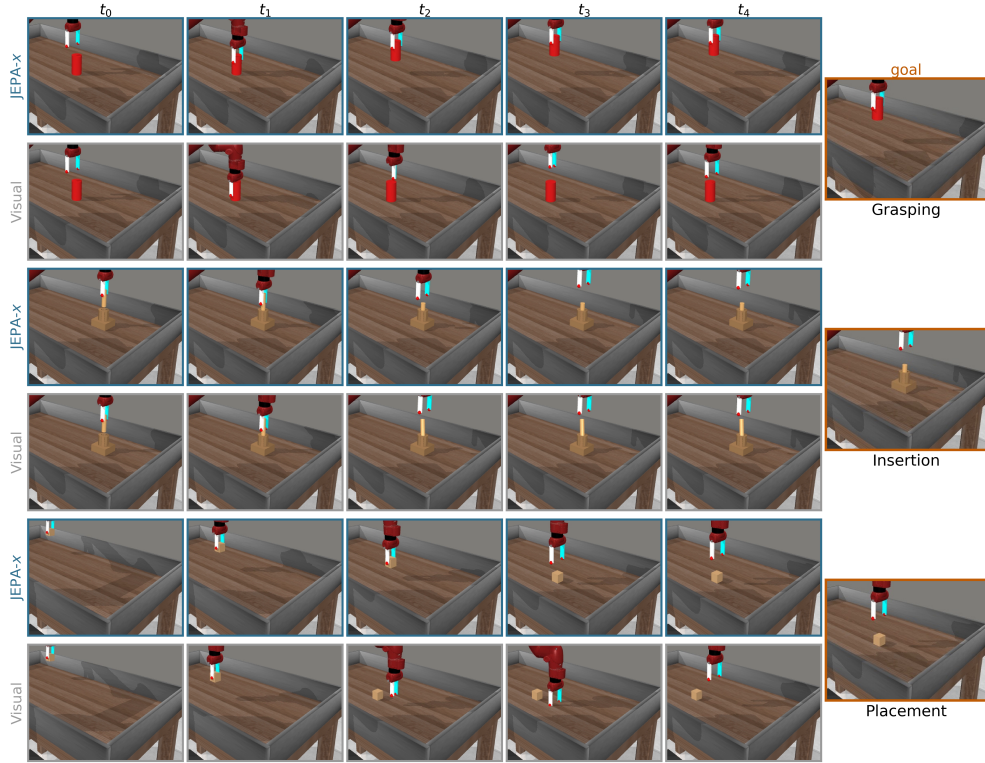


Figure 7: **JEPa-x against VISUAL on the same scenario: grasping, insertion, and placement.** Each family contributes a pair of rows executed from the same initial state using the same planner, JEPa-x above and VISUAL below, cropped to the working volume. The goal is shared by both arms and is therefore shown once per comparison, spanning the family’s two rows. Scenarios are the representative episodes in which the two arms disagree, so each pair is a case JEPa-x completes and VISUAL does not; per-family success rates are in Table 6.

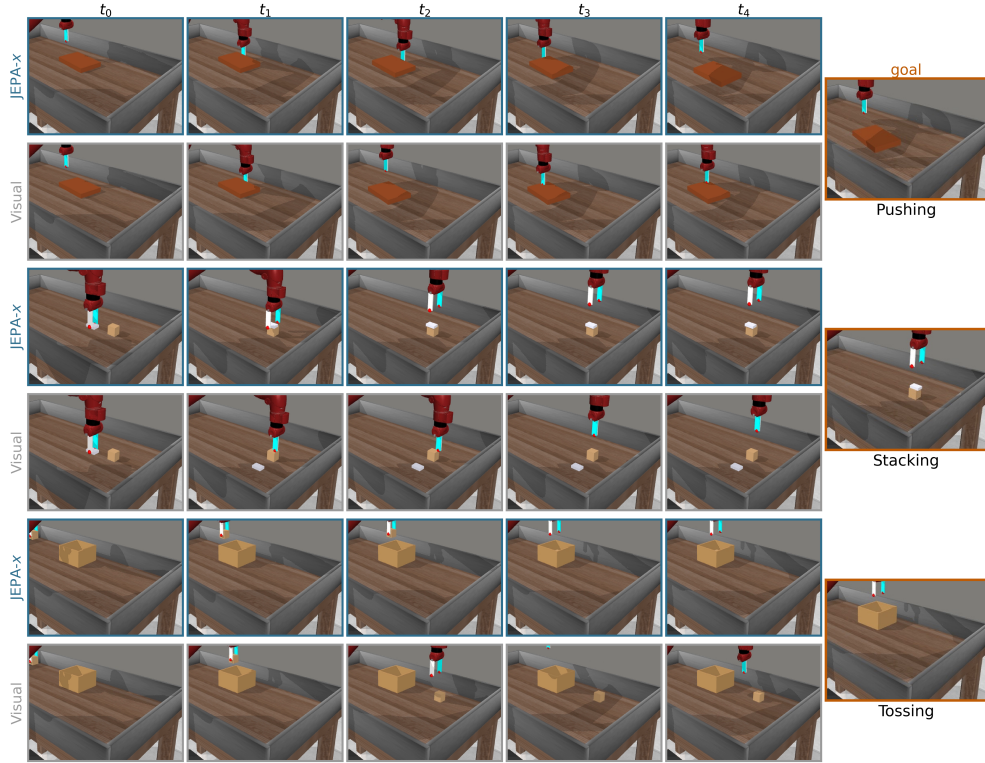


Figure 8: **JEPA-x against VISUAL on the same scenario: pushing, stacking, and tossing.** Same protocol as Figure 7: paired rows from one initial state, JEPA-x above and VISUAL below, with the shared goal shown once per comparison. In stacking, VISUAL leaves the eraser beside the block; in tossing, it leaves the object outside the carton.